

**Remotely Accessible Cyber-Physical System
Testbed and Open Architecture Synchrophasor
Systems for Bangladesh's National Power Grid's
Cyber Security and Reliability**

Cyber-Physical System Testbed

Co-Simulation of the IEEE 14-Bus System
GridPACK • HELICS • NS-3 • Node-RED

Technical Report

Prepared by

Md. Mazharul Islam

Table of Contents

1. Introduction.....	4
2. System Architecture.....	4
2.1 Components.....	4
2.2 Data flow.....	5
3. Component Operation.....	5
3.1 GridPACK: Power System Layer.....	5
3.2 NS-3: Communication Layer.....	6
3.3 HELICS: Co-Simulation Engine.....	7
3.4 Node-RED: Visualization Layer.....	8
4. Implementation Environment.....	9
5. Running the Testbed.....	9
6. Results.....	12
6.1 Normal Mode (No Loss, No Attack).....	12
6.2 Lost Packet (Communication-Layer Failure).....	16
6.3 Cyber Attack - False Data Injection.....	17
7. Extended Testbed: Multi-PMU Co-Simulation.....	22
7.1 Architecture.....	22
7.2 Live GridPACK Power Flow and Bus Systems.....	23
7.3 Optimal PMU Placement.....	24
7.4 Communication Network.....	24
7.5 Control Center: State Estimation, Bad Data Detection and Control.....	24
7.6 Web Application.....	25
7.7 Deployment and Container Images.....	25
8. Extended Results.....	26
8.1 Scalability Across Bus Systems.....	26
8.2 Closed-Loop Voltage Control.....	26
8.3 Network Latency and the PDC Wait Window.....	27

8.4 Attacks and Defenses.....	27
8.5 Critical Measurements and Redundant PMU Placement.....	29
9. Summary.....	29

1. Introduction

This report describes a Cyber-Physical System (CPS) testbed built to study the interaction between the electrical behavior of a power grid and the communication network that carries its measurements and control signals. The testbed targets the research goal of the NSU-PGCB project: providing a controlled, simulation-based environment in which synchrophasor applications, network conditions, and cyber-security scenarios can be tested safely before any deployment on the live national grid.

The present National Load Dispatch Center relies on SCADA, which polls data every 2 to 5 seconds and is not time-synchronized. The testbed instead models a synchrophasor-style data path in which grid measurements are produced, transported over a simulated network, and consumed by a concentrator, with the timing and integrity of that path observable and controllable.

The testbed couples two independent simulators one for the power system and one for the communication network - through a co-simulation engine that keeps them time-synchronized. A dashboard presents the results to an operator. Here, I am using IEEE 14-bus system is used as the reference power network throughout.

2. System Architecture

The testbed is composed of four software components. Two are simulators, one is the synchronization engine that connects them, and one is the visualization layer. The two simulators never exchange data directly; all exchange passes through the co-simulation engine, which is the defining characteristic of the design.

2.1 Components

The four components, their roles, and the layer each represents are summarized in Table 1.

Component	Role in the testbed	Layer
GridPACK	Solves the IEEE 14-bus power flow and produces bus voltages and angles.	Power system
NS-3	Models the communication network carrying measurements from a PMU node to a PDC node.	Communication
HELICS	Synchronizes the two simulators in time and carries data between them as a federation.	Co-simulation
Node-RED	Reads the result files and presents an operator dashboard.	Visualization

Table 1: The four components of the CPS testbed and their roles.

2.2 Data flow

A single run proceeds as follows. GridPACK publishes each bus voltage into the HELICS federation. NS-3, joined to the same federation, receives each voltage at a simulated PMU node and transmits it as a UDP packet across a point-to-point link of configurable latency to a simulated PDC node. The PDC inspects each received measurement; if a voltage exceeds the safety limit, the PDC publishes a control command back through HELICS to GridPACK, which receives and acknowledges it. This forms a closed monitoring-and-control loop. NS-3 writes the per-bus results and a run summary to disk, and Node-RED reads those files to render the dashboard. The control flow of the CPS testbed is illustrated in Fig. 1.

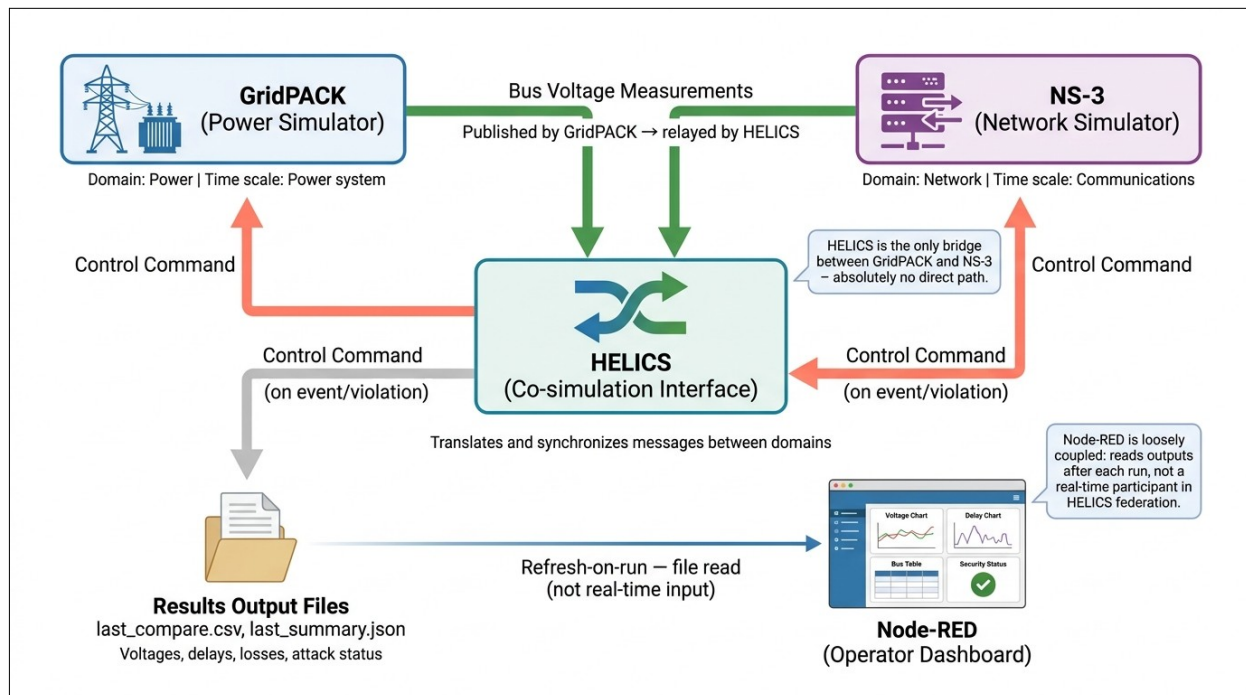


Fig 1: End-to-end data flow in the CPS testbed

The path can be summarized as: GridPACK → HELICS → NS-3 (PMU → network → PDC) → HELICS → GridPACK, with NS-3 writing result files that Node-RED reads.

3. Component Operation

3.1 GridPACK: Power System Layer

GridPACK solves the steady-state power flow of the IEEE 14-bus network. For each of the 14 buses it produces a voltage magnitude (per unit) and a phase angle (degrees). These values match the standard IEEE 14-bus benchmark; for example, bus 1 is the slack bus at 1.06 pu and 0 degrees, and bus 8 sits at 1.09 pu. The solved values are exported to a CSV file and republished into the federation by a GridPACK federate so that the network simulator can consume them step by step. A representative power-flow solution for the IEEE 14-bus system produced by GridPACK is shown in Fig. 2.

```
(cps-env-ubuntu) ubuntu@cps:~/software/ns-3$ echo "==== GRIDPACK: IEEE 14-bus power flow solution ====="
cat ~/cps-work/pf14/pf_IEEE14_buses.csv
==== GRIDPACK: IEEE 14-bus power flow solution =====
bus_id,type,area,zone,base_kv,voltage_pu,angle_deg,p_injection_mw,q_injection_mvar,p_load_mw,q_load_mvar,p_gen_mw,q_gen
mvar,shunt_mvar
1,3,1,2,100.00,1.060000,0.000000,232.3933,-16.9000,0.0000,0.0000,232.3933,-16.9000,0.0000
2,2,1,2,100.00,1.045000,-4.982589,18.3000,30.8571,21.7000,12.7000,40.0000,43.5571,0.0000
3,2,1,2,100.00,1.010000,-12.725100,-94.2000,6.0753,94.2000,19.0000,0.0000,25.0753,0.0000
4,1,1,2,100.00,1.017671,-10.312901,-47.8000,3.9000,47.8000,-3.9000,0.0000,0.0000,0.0000
5,1,1,2,100.00,1.019514,-8.773854,-7.6000,-1.6000,7.6000,1.6000,0.0000,0.0000,0.0000
6,2,1,2,100.00,1.070000,-14.220946,-11.2000,5.2309,11.2000,7.5000,0.0000,12.7309,0.0000
7,1,1,2,100.00,1.061520,-13.359627,0.0000,0.0000,0.0000,0.0000,0.0000,0.0000,0.0000
8,2,1,2,100.00,1.090000,-13.359627,0.0000,17.6235,0.0000,0.0000,0.0000,17.6235,0.0000
9,1,1,2,100.00,1.055932,-14.938521,-29.5000,-16.6000,29.5000,16.6000,0.0000,0.0000,19.0000
10,1,1,2,100.00,1.050985,-15.097288,-9.0000,-5.8000,9.0000,5.8000,0.0000,0.0000,0.0000
11,1,1,2,100.00,1.056907,-14.790622,-3.5000,-1.8000,3.5000,1.8000,0.0000,0.0000,0.0000
12,1,1,2,100.00,1.055189,-15.075585,-6.1000,-1.6000,6.1000,1.6000,0.0000,0.0000,0.0000
13,1,1,2,100.00,1.050382,-15.156276,-13.5000,-5.8000,13.5000,5.8000,0.0000,0.0000,0.0000
14,1,1,2,100.00,1.035530,-16.033645,-14.9000,-5.0000,14.9000,5.0000,0.0000,0.0000,0.0000
(cps-env-ubuntu) ubuntu@cps:~/software/ns-3$
```

Fig 2: Voltage magnitude and phase angle results obtained from the IEEE 14-bus power-flow solution in GridPACK.

3.2 NS-3: Communication Layer

NS-3 constructs a two-node network: node 0 acts as a PMU (the measurement source) and node 1 acts as a PDC (the phasor data concentrator). They are connected by a point-to-point link whose one-way latency is a run parameter. Each voltage received from the federation is encoded into a UDP packet and sent across the link.

The PDC records the arrival time of every packet, computes the end-to-end delay, checks the received voltage against the safety limit, and when warranted raises a control command. The network can also be configured to drop packets at a chosen rate to study the effect of loss on grid observability. The NS-3 communication network used in the co-simulation is illustrated in Fig. 3.

```
(cps-env-ubuntu) ubuntu@cps:~/software/ns-3$ echo "==== NS-3: network model (key parts) ====="
grep -n -A1 "nodes.Create\|PointToPoint\|SetChannelAttribute\|PMU\|PDC\|registerSubscription\|registerGlobalPublication"
~/software/ns-3/scratch/helictest/helictest.cc | head -40
==== NS-3: network model (key parts) =====
64: // PDC decides based on what it RECEIVED (possibly tampered)
65: if (v > g_vLimit) {
--
68:     std::cout << "[PDC] bus " << bus << " V=" << v
69:     << " -> VIOLATION detected, command raised" << std::endl;
--
93: auto sub = fed->registerSubscription("gridpack/bus1_voltage");
94: auto cmdPub = fed->registerGlobalPublication<std::string>("ns3/control_command");
95:
96: NodeContainer nodes; nodes.Create(2);
97: PointToPointHelper p2p;
98: p2p.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
99: p2p.SetChannelAttribute("Delay", TimeValue(MilliSeconds(latencyMs)));
100: NetDeviceContainer devs = p2p.Install(nodes);
--
189: << " \ns3\": \nPMU->PDC UDP over point-to-point\n"
190: << "\n";
```

Fig 3: NS-3 communication model consisting of a PMU node, a point-to-point network link, and a PDC node.

3.3 HELICS: Co-Simulation Engine

HELICS is the broker that binds the two simulators into a single federation and advances their clocks together. GridPACK and NS-3 are separate simulators processes written in different languages; they share no memory and no files for live data. HELICS is the only channel between them. Each component publishes named values and subscribes to others: GridPACK publishes the bus voltage and subscribes to the control command; NS-3 subscribes to the voltage and publishes the command.

The broker guarantees that a value produced at a given logical time is delivered at the correct corresponding time on the other side. The role of HELICS in synchronizing the power-system and communication-network simulators is illustrated in Fig. 4.

```
[PMU-node0] sent bus=1 V=1.06 into network
[PDC-node1] received bus=1 V=1.06 (delay 10.0336 ms)
[PMU-node0] sent bus=2 V=1.045 into network
[PDC-node1] received bus=2 V=1.045 (delay 10.0344 ms)
[PMU-node0] sent bus=3 V=1.01 into network
[PDC-node1] received bus=3 V=1.01 (delay 10.0336 ms)
[PMU-node0] sent bus=4 V=1.01767 into network
[PDC-node1] received bus=4 V=1.01767 (delay 10.036 ms)
[PMU-node0] sent bus=5 V=1.01951 into network
[PDC-node1] received bus=5 V=1.01951 (delay 10.036 ms)
[PMU-node0] sent bus=6 V=1.07 into network
[PDC-node1] received bus=6 V=1.07 (delay 10.0336 ms)
[PMU-node0] sent bus=7 V=1.06152 into network
[PDC-node1] received bus=7 V=1.06152 (delay 10.036 ms)
[PMU-node0] sent bus=8 V=1.09 into network
[PDC-node1] received bus=8 V=1.09 (delay 10.0336 ms)
[CONTROL] command -> GridPACK: REDUCE_VOLTAGE@bus8
[PMU-node0] sent bus=9 V=1.05593 into network
[PDC-node1] received bus=9 V=1.05593 (delay 10.036 ms)
[PMU-node0] sent bus=10 V=1.05099 into network
[PDC-node1] received bus=10 V=1.05099 (delay 10.0368 ms)
[PMU-node0] sent bus=11 V=1.05691 into network
[PDC-node1] received bus=11 V=1.05691 (delay 10.0368 ms)
[PMU-node0] sent bus=12 V=1.05519 into network
[PDC-node1] received bus=12 V=1.05519 (delay 10.0368 ms)
[PMU-node0] sent bus=13 V=1.05038 into network
[PDC-node1] received bus=13 V=1.05038 (delay 10.0368 ms)
[PMU-node0] sent bus=14 V=1.03553 into network
[PDC-node1] received bus=14 V=1.03553 (delay 10.0368 ms)
[GRIDPACK] federation joined, publishing voltages + listening for control commands
[GRIDPACK] t=1.0s bus 1 V=1.0600 pu angle=0.00 deg
[GRIDPACK] t=1.0s bus 2 V=1.0450 pu angle=-4.98 deg
[GRIDPACK] t=2.0s bus 3 V=1.0100 pu angle=-12.73 deg
[GRIDPACK] t=3.0s bus 4 V=1.0177 pu angle=-10.31 deg
[GRIDPACK] t=4.0s bus 5 V=1.0195 pu angle=-8.77 deg
[GRIDPACK] t=5.0s bus 6 V=1.0700 pu angle=-14.22 deg
[GRIDPACK] t=6.0s bus 7 V=1.0615 pu angle=-13.36 deg
[GRIDPACK] t=7.0s bus 8 V=1.0900 pu angle=-13.36 deg
[GRIDPACK] t=8.0s bus 9 V=1.0559 pu angle=-14.94 deg
[GRIDPACK] *** CONTROL COMMAND RECEIVED: REDUCE_VOLTAGE@bus8 -> applying corrective action at this bus ***
[GRIDPACK] t=9.0s bus 10 V=1.0510 pu angle=-15.10 deg
[GRIDPACK] t=10.0s bus 11 V=1.0569 pu angle=-14.79 deg
[GRIDPACK] t=11.0s bus 12 V=1.0552 pu angle=-15.08 deg
[GRIDPACK] t=12.0s bus 13 V=1.0504 pu angle=-15.16 deg
```

Fig 4: HELICS federation connecting GridPACK and NS-3 through synchronized publish–subscribe communication.

3.4 Node-RED: Visualization Layer

Node-RED reads the two result files that NS-3 writes a per-bus CSV and a run-summary JSON and renders an operator console in the browser. The console shows the true versus delivered voltage profile, the per-bus network delay, a measurement table with status flags, and a summary panel reporting delivery, loss, delay and any detected violations.

Node-RED is loosely coupled: it consumes the files produced by a run rather than joining the HELICS federation, so the dashboard refreshes on each run rather than streaming continuously. The structure of the dashboard and the underlying flow that produces it are shown in Fig. 5 and Fig. 6 respectively.

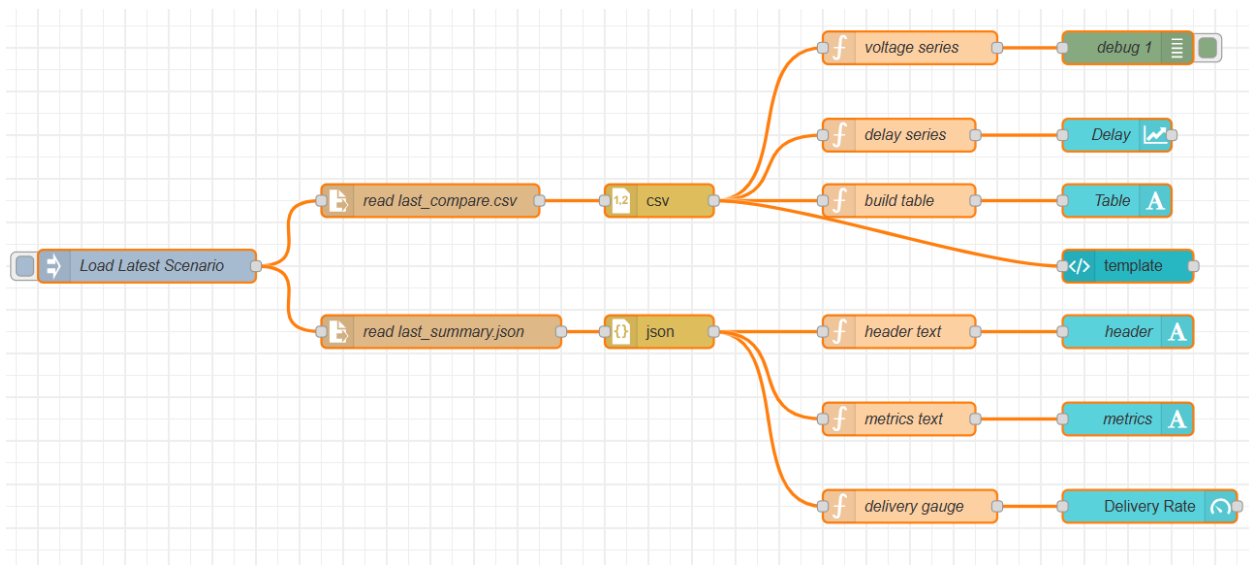


Fig 5. The Node-RED operator console as seen in the browser.

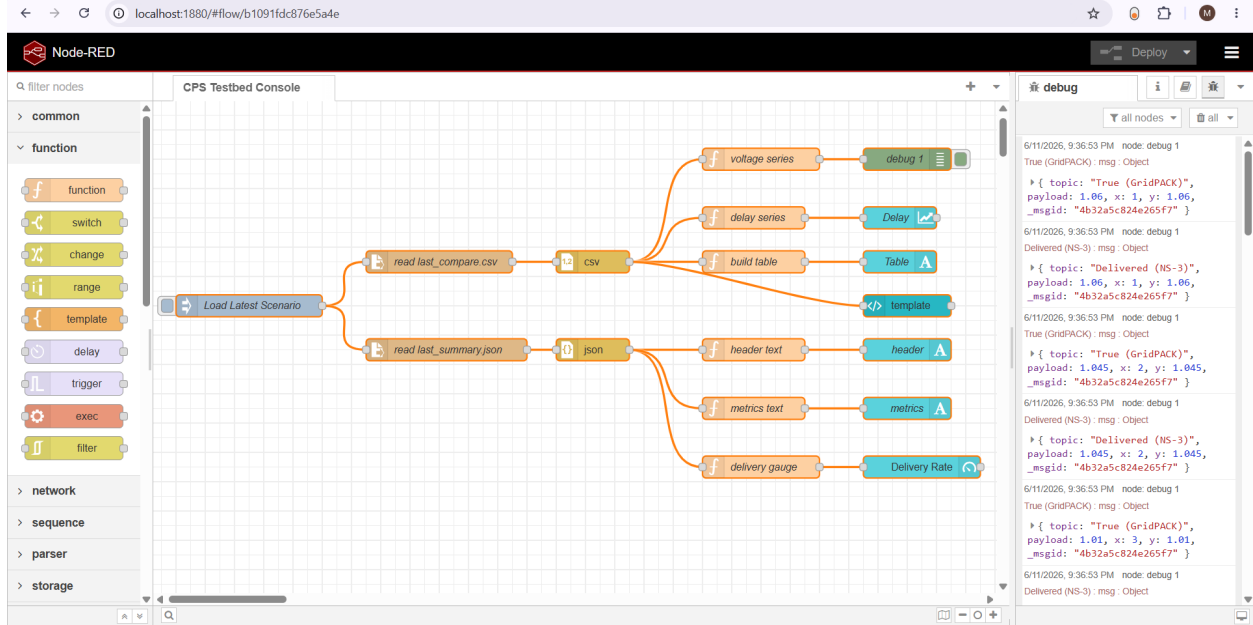


Fig 6: The underlying Node-RED flow in the editor.

4. Implementation Environment

All components were built from source on a single Ubuntu 24.04 server. The principal versions and locations are listed in Table 2.

Item	Detail
Operating system	Ubuntu 24.04, 8 cores, 32 GB RAM
GridPACK	Built from source; IEEE 14-bus power-flow application
HELICS	Version 3.6.1 (C++ build with shared library)
NS-3	ns-3-dev (CMake build) with a ported HELICS module
Node-RED	Version 4.1.11 with FlowFuse Dashboard 2.0

Table 2: The components of the CPS testbed Implementation.

5. Running the Testbed

A scenario controller launches all federates with a single command and saves the results. The network latency, packet-loss rate, attack flag, and scenario name are all parameters.

```
./run_scenario.sh --latency=20 --loss=0.2 --name=demo
```

Each run starts the HELICS broker, the GridPACK federate, and the NS-3 network federate, runs the co-simulation, and writes two output files: a per-bus comparison CSV (true voltage, received voltage, delay, delivered flag) and a summary JSON (scenario, latency, delivered and lost counts,

average and maximum delay, and violations). A representative execution of the scenario controller and the resulting co-simulation workflow are shown in Fig. 7.

```
ubuntu@cps:~/cps-testbed$ ./run_scenario.sh --latency=20 --loss=0.2 --name=demo
=====
CPS Testbed Scenario: demo
Network latency: 20 ms
Log: /home/ubuntu/cps-testbed/results/demo_20260611_165659.log
=====
[0/2] Re-checking globbed directories...
ninja: no work to do.
[CPS] scenario=demo latency=20ms loss=20% attack=OFF
[PMU-node0] sent bus=1 V=1.06 into network
[PDC-node1] received bus=1 V=1.06 (delay 20.0336 ms)
[PMU-node0] sent bus=2 V=1.045 into network
[PDC-node1] received bus=2 V=1.045 (delay 20.0344 ms)
[PMU-node0] sent bus=3 V=1.01 into network
[PDC-node1] received bus=3 V=1.01 (delay 20.0336 ms)
[PMU-node0] sent bus=4 V=1.01767 into network
[PDC-node1] received bus=4 V=1.01767 (delay 20.036 ms)
[PMU-node0] sent bus=5 V=1.01951 into network
[PDC-node1] received bus=5 V=1.01951 (delay 20.036 ms)
[PMU-node0] sent bus=6 V=1.07 into network
[PDC-node1] received bus=6 V=1.07 (delay 20.0336 ms)
[PMU-node0] sent bus=7 V=1.06152 into network
[PMU-node0] sent bus=7 V=1.06152 into network
[PMU-node0] sent bus=8 V=1.09 into network
[PDC-node1] received bus=8 V=1.09 (delay 20.0336 ms)
[PDC] bus 8 V=1.09 -> VIOLATION detected, command raised
[CONTROL] command -> GridPACK: REDUCE_VOLTAGE@bus8
[PMU-node0] sent bus=9 V=1.05593 into network
[PDC-node1] received bus=9 V=1.05593 (delay 20.036 ms)
[PMU-node0] sent bus=10 V=1.05099 into network
[PDC-node1] received bus=10 V=1.05099 (delay 20.0368 ms)
[PMU-node0] sent bus=11 V=1.05691 into network
[PDC-node1] received bus=11 V=1.05691 (delay 20.0368 ms)
[PMU-node0] sent bus=12 V=1.05519 into network
[PDC-node1] received bus=12 V=1.05519 (delay 20.0368 ms)
[PMU-node0] sent bus=13 V=1.05038 into network
[PDC-node1] received bus=13 V=1.05038 (delay 20.0368 ms)
[GRIDPACK] federation joined, publishing voltages + listening for control commands
[GRIDPACK] t=1.0s bus 1 V=1.0600 pu angle=0.00 deg
[GRIDPACK] t=1.0s bus 2 V=1.0450 pu angle=-4.98 deg
[GRIDPACK] t=2.0s bus 3 V=1.0100 pu angle=-12.73 deg
[GRIDPACK] t=3.0s bus 4 V=1.0177 pu angle=-10.31 deg
[GRIDPACK] t=4.0s bus 5 V=1.0195 pu angle=-8.77 deg
[GRIDPACK] t=5.0s bus 6 V=1.0700 pu angle=-14.22 deg
[GRIDPACK] t=6.0s bus 7 V=1.0615 pu angle=-13.36 deg
[GRIDPACK] t=7.0s bus 8 V=1.0900 pu angle=-13.36 deg
[GRIDPACK] t=8.0s bus 9 V=1.0559 pu angle=-14.94 deg
[GRIDPACK] *** CONTROL COMMAND RECEIVED: REDUCE_VOLTAGE@bus8 -> applying corrective action at
this bus ***
[GRIDPACK] t=9.0s bus 10 V=1.0510 pu angle=-15.10 deg
[GRIDPACK] t=10.0s bus 11 V=1.0569 pu angle=-14.79 deg
[GRIDPACK] t=11.0s bus 12 V=1.0552 pu angle=-15.08 deg
[GRIDPACK] t=12.0s bus 13 V=1.0504 pu angle=-15.16 deg
[GRIDPACK] t=13.0s bus 14 V=1.0355 pu angle=-16.03 deg
[GRIDPACK] finalized
[PMU-node0] sent bus=14 V=1.03553 into network
[PDC-node1] received bus=14 V=1.03553 (delay 20.0368 ms)
[CPS] SUMMARY: delivered 13/14, violations seen 1, MISSED violations (hidden by attack) 0
```

Fig 7: Launching a CPS co-simulation scenario using the automated scenario controller.

5.1 End-to-End Data Path Demonstration

To verify that the three core components operate together as a single synchronized system, the testbed can be run in a demonstration mode in which each stage prints its own output. This makes the complete cyber-physical data path visible, from the power-system calculation, through the communication network, to the concentrator and back. A single bus measurement can be traced through the entire loop.

For bus 8, which carries a true voltage of 1.09 pu, the sequence observed in a representative run is as follows:

- GridPACK first computes the value as part of the IEEE 14-bus power-flow solution and publishes it into the federation.
- The PMU node in NS-3 receives this value and transmits it across the simulated network as a UDP packet.
- The PDC node receives the packet after the configured link delay, recovers the voltage, and because the value exceeds the 1.08 pu safety limit raises a control command.
- That command is published back through HELICS to GridPACK, which receives it and acknowledges a corrective action.
- The corresponding console output for this single measurement is:

```
GridPACK: bus 8, V = 1.0900 pu, angle = -13.36 deg PMU (node 0): sent bus 8, V = 1.09  
into network PDC (node 1): received bus 8, V = 1.09, delay 10.03 ms Control: command to  
GridPACK, REDUCE_VOLTAGE@bus8 GridPACK: control command received,  
applying corrective action
```

This trace confirms the full closed loop. The measurement path carries the voltage from the power simulator to the concentrator (GridPACK → PMU → network → PDC), and the control path returns the command from the concentrator to the power simulator (PDC → HELICS → GridPACK). The two paths together form the complete monitoring-and-control cycle, with all 14 buses delivered and the single violation at bus 8 correctly detected and acted upon.

It should be noted that GridPACK and NS-3 run as two independent simulators processes on different logical time scales — GridPACK advances in one-second power-system steps, while NS-3 resolves network events at the sub-millisecond level. As a result, the interleaving of their console messages is not aligned in real time, and the printed lines from one process may appear before or after those of the other.

The data synchronization through HELICS is nonetheless exact: each value produced at a given logical time is delivered to the correct counterpart at the corresponding time, as confirmed by the fact that the correct voltage reaches the PDC and the resulting command returns to the correct bus. The verbose per-stage printing is intended for demonstration; for routine scenario runs the same

information is recorded to the result files, and the printing can be disabled to keep output clean and runs fast.

6. Results

The testbed was exercised under three operating scenarios that together demonstrate its purpose: a normal baseline, a communication-layer failure (packet loss), and a cyber-security failure (a false-data-injection attack). In every scenario the blue line is the true grid state computed by GridPACK, and the red step line is the voltage actually delivered to the PDC across the simulated network. The contrast between the three cases shows how the testbed distinguishes a healthy data path, a network fault, and a data-integrity attack.

6.1 Normal Mode (No Loss, No Attack)

In the baseline scenario the network is healthy: no packets are lost and no measurement is tampered with. Every one of the 14 bus voltages travels intact from GridPACK, through the PMU and the simulated link, to the PDC, delayed only by the configured network latency. As shown in Fig. 8, the delivered profile (red) lies exactly on top of the true profile (blue) at all 14 buses, and the per-bus status reads OK throughout. This is the testbed operating as a faithful, transparent data path, and it is the reference against which the two fault scenarios are measured.

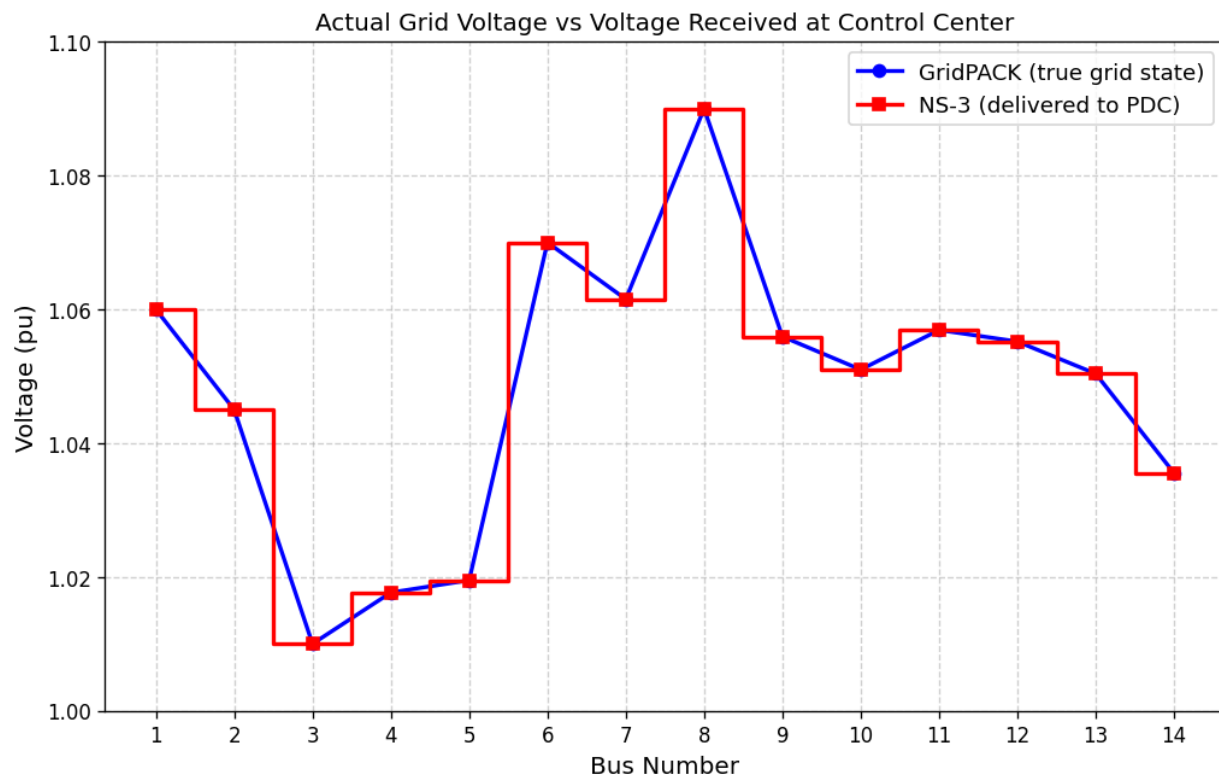


Fig. 8. Actual grid voltage versus voltage received at the control center under normal operation.

A key property of this mode is that the network latency is not merely cosmetic — it genuinely shapes the data path. The end-to-end delay measured at the PDC tracks the configured link latency closely and changes with it, as summarized in Table 1. For link settings of 5, 20, and 200 ms, the measured per-bus delay is approximately 5.03, 20.04, and 200.03 ms respectively, with all 14 measurements delivered in every case. This confirms that the network model actively transports the data rather than passing it through unchanged, and it establishes the delay floor against which loss and attack scenarios are later compared.

Configured latency	Measured delay (per bus)	Delivered
5 ms	~5.03 ms	14 / 14
20 ms	~20.04 ms	14 / 14
200 ms	~200.03 ms	14 / 14

Table 3: Measured end-to-end delay versus configured link latency, normal mode.

The full operator dashboard for this normal-mode run is shown in Fig. 9.

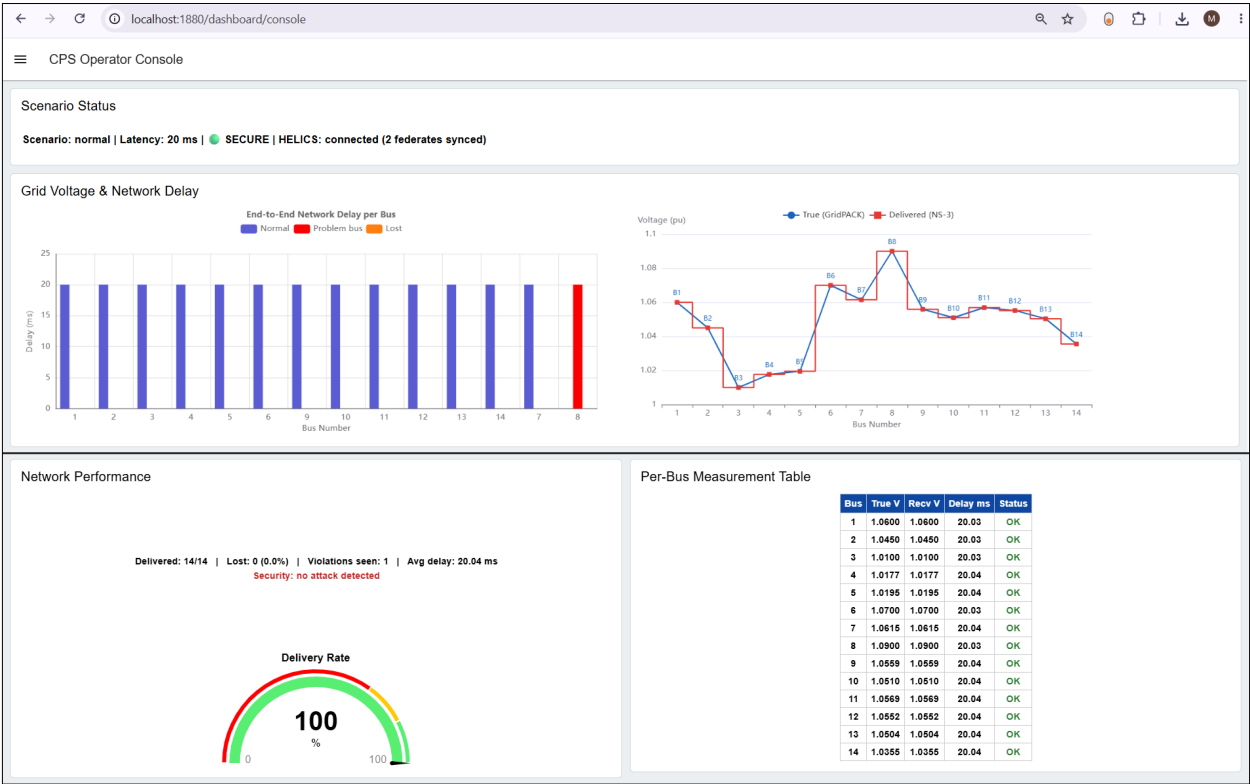


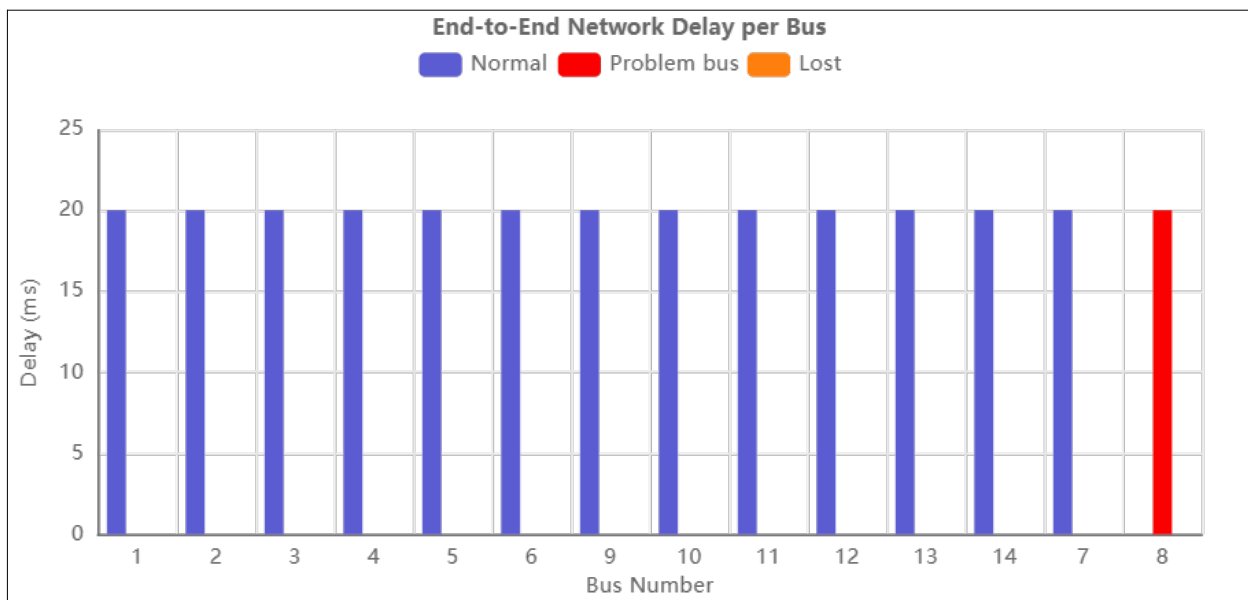
Fig 9: The complete operator dashboard under normal operation — all 14 buses delivered, no loss, no attack.

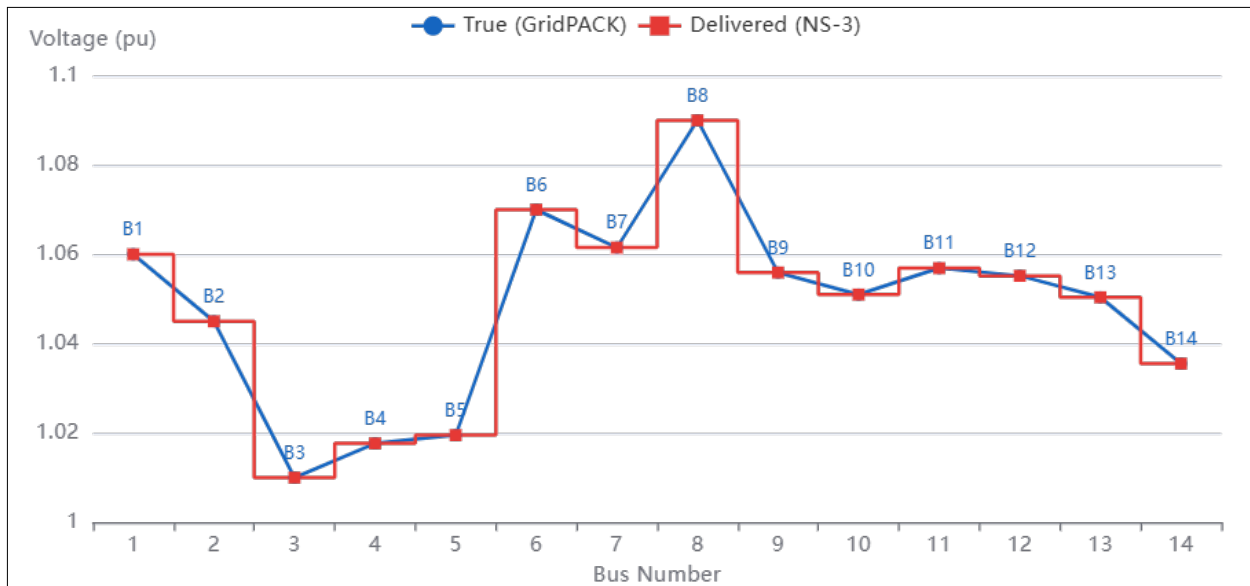
Each panel of the dashboard is shown individually below: the grid voltage comparison, the network delay, the network performance summary, and the per-bus measurement table.

≡ CPS Operator Console

Scenario Status

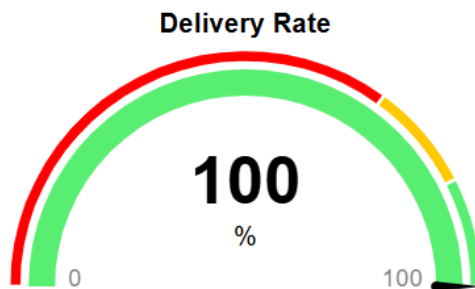
Scenario: normal | Latency: 20 ms | ● SECURE | HELICS: connected (2 federates synced)





Network Performance

Delivered: 14/14 | Lost: 0 (0.0%) | Violations seen: 1 | Avg delay: 20.04 ms
Security: no attack detected



Per-Bus Measurement Table

Bus	True V	Recv V	Delay ms	Status
1	1.0600	1.0600	20.03	OK
2	1.0450	1.0450	20.03	OK
3	1.0100	1.0100	20.03	OK
4	1.0177	1.0177	20.04	OK
5	1.0195	1.0195	20.04	OK
6	1.0700	1.0700	20.03	OK
7	1.0615	1.0615	20.04	OK
8	1.0900	1.0900	20.03	OK
9	1.0559	1.0559	20.04	OK
10	1.0510	1.0510	20.04	OK
11	1.0569	1.0569	20.04	OK
12	1.0552	1.0552	20.04	OK
13	1.0504	1.0504	20.04	OK
14	1.0355	1.0355	20.04	OK

6.2 Lost Packet (Communication-Layer Failure)

In the second scenario, network packet loss is introduced, and one measurement fails to reach the concentrator. As shown in Fig. 10, bus 7 - an electrically healthy bus at 1.06 pu - is dropped by the network in transit and is flagged LOST on the chart. The loss has nothing to do with the grid: the bus itself is perfectly normal, but its packet simply never arrives, so the PDC has no value for it. The result is a blind spot in the operator's view. Because the delivered profile cannot fill in the missing point, it bridges across bus 7, while the true profile continues uninterrupted.

The lesson for the control center is that a missing measurement is not the same as a healthy reading: the operator must be able to distinguish "no data" from "good data," because packet loss can conceal parts of the grid even when nothing is physically wrong.

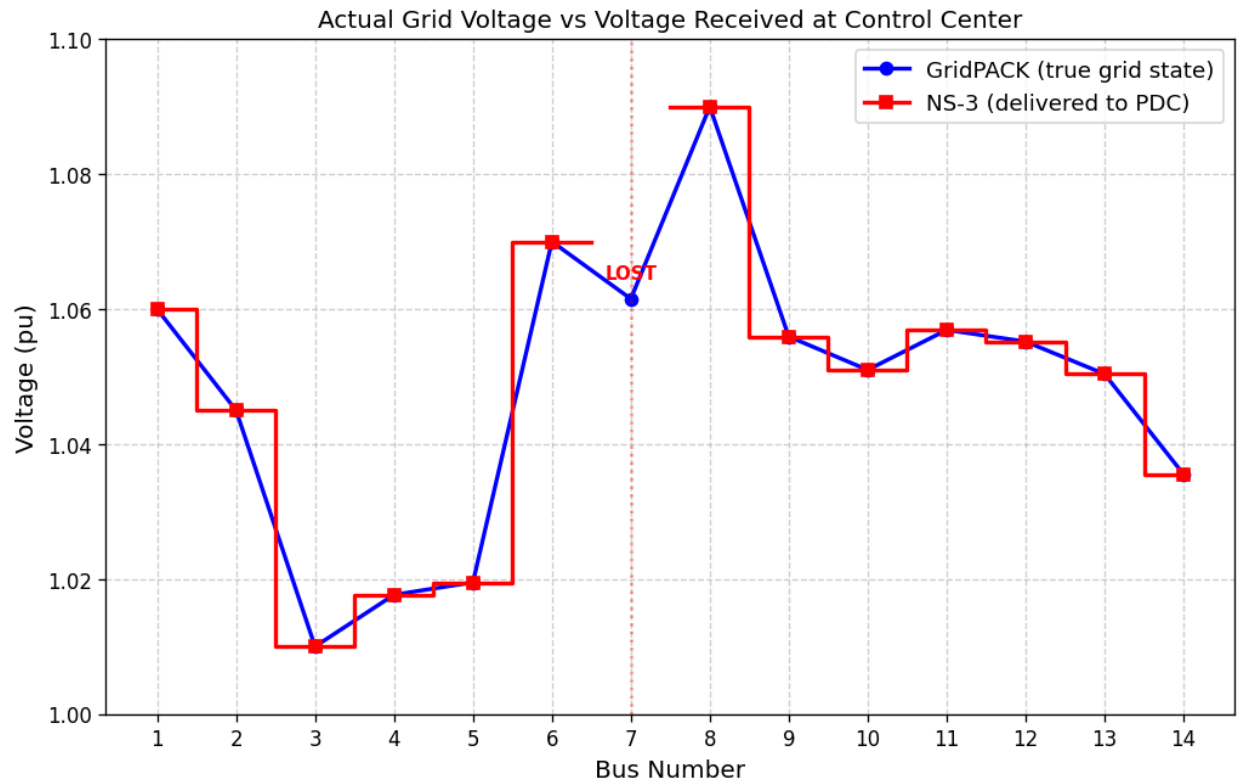


Fig. 10. A lost packet at bus 7.

When packet loss is enabled, some measurements never reach the PDC. In a representative run with a 20 percent loss setting, 13 of 14 voltages were delivered and one bus was lost. On the dashboard the delivered profile diverges from the true profile at the lost bus, illustrating how loss degrades the operator's view of the grid.

6.3 Cyber Attack - False Data Injection

The third and most serious scenario is a data-integrity attack. Bus 8 carries a true voltage of 1.09 pu, above the 1.08 pu safety limit - a genuine violation that the PDC would normally detect and act upon. Under a false-data-injection attack, an adversary intercepts the bus 8 measurement in transit and rewrites it, replacing the true 1.09 pu with a safe-looking 1.05 pu before it reaches the concentrator.

As shown in Fig. 11, the true profile (blue) spikes sharply to 1.09 pu at bus 8, while the delivered profile (red) remains flat at 1.05 pu - the visible signature of the forgery.

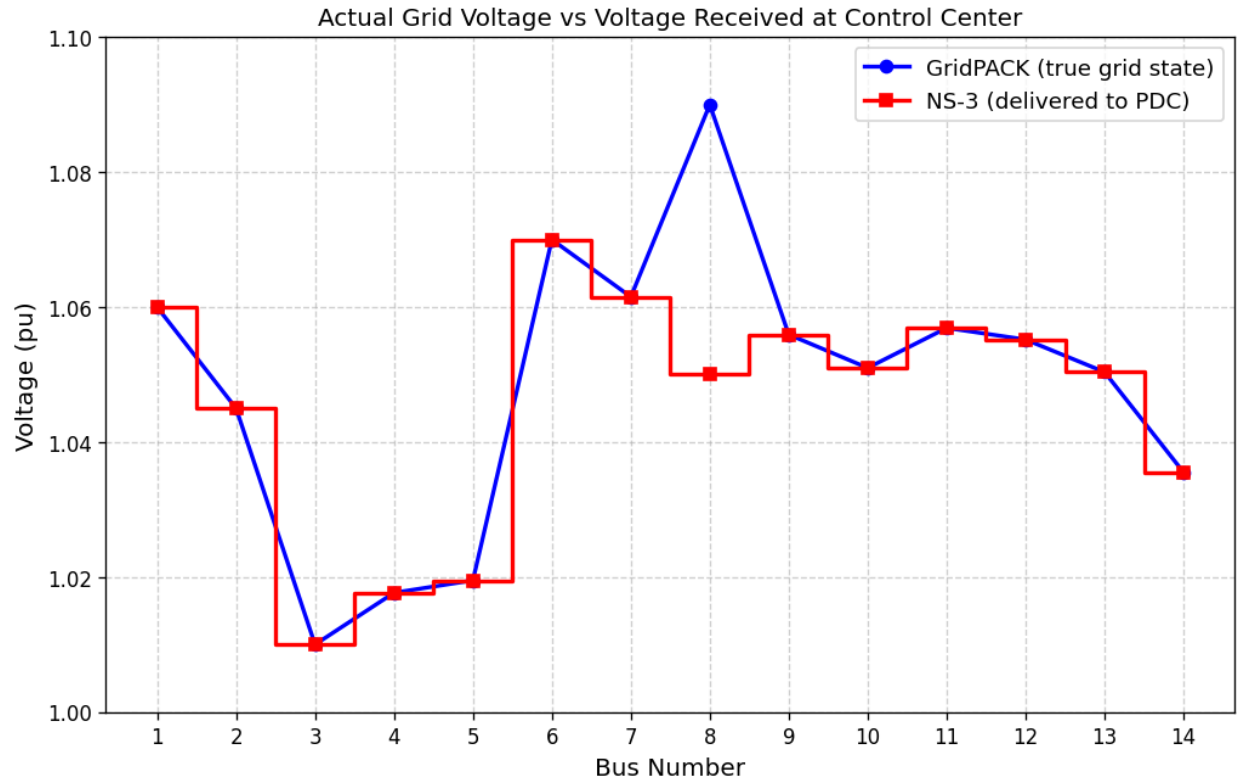


Fig 11: Hidden violation at bus 8 under a false-data-injection attack.

The PDC therefore sees a normal value, raises no alarm, and issues no corrective command, even though the real overvoltage is still present on the grid. Unlike the lost packet of the previous scenario, here the data does arrive - but it is a lie. The contrast is summarized in Table 4.

Metric	Normal run	Under attack
Bus 8 true voltage	1.09 pu	1.09 pu
Value seen by PDC	1.09 pu	1.05 pu (forged)
Violation detected	Yes	No
Control command sent	Yes	No

Missed violations	0	1
-------------------	---	---

Table 4: Effect of the false-data-injection attack on bus 8: normal run versus attack.

This is the central cyber-security result of the testbed: a concrete demonstration of how a stealthy false-data-injection attack can defeat a synchrophasor-based protection scheme by making a genuine grid violation invisible to the control center. Because the forged value passes every basic check the concentrator performs, the attack succeeds silently — which is precisely why such threats are dangerous and why a testbed that can reproduce them, safely and repeatably, is valuable before any deployment on the live grid. The full operator dashboard for this cyber-attack mode run is shown in Fig. 12.

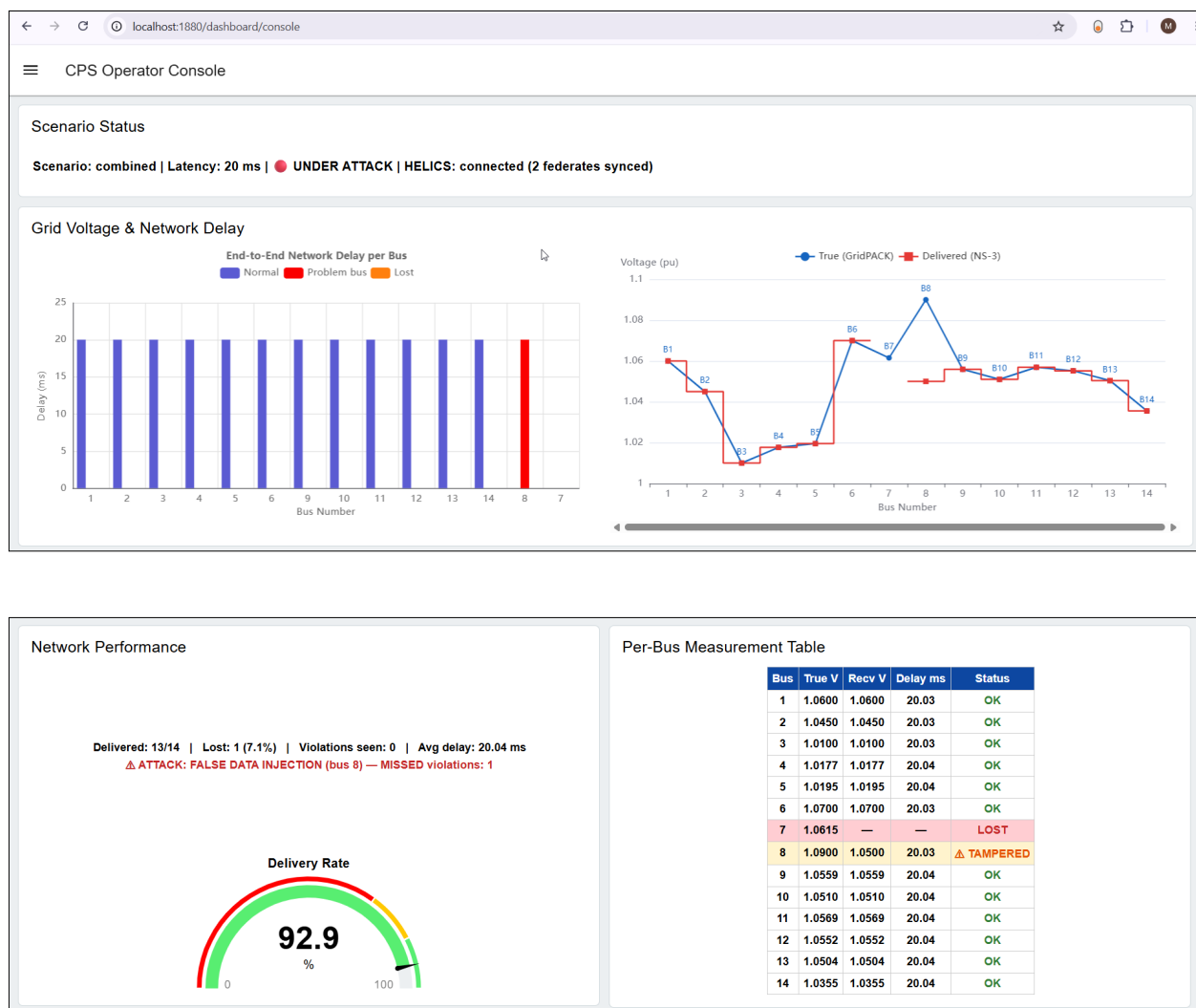
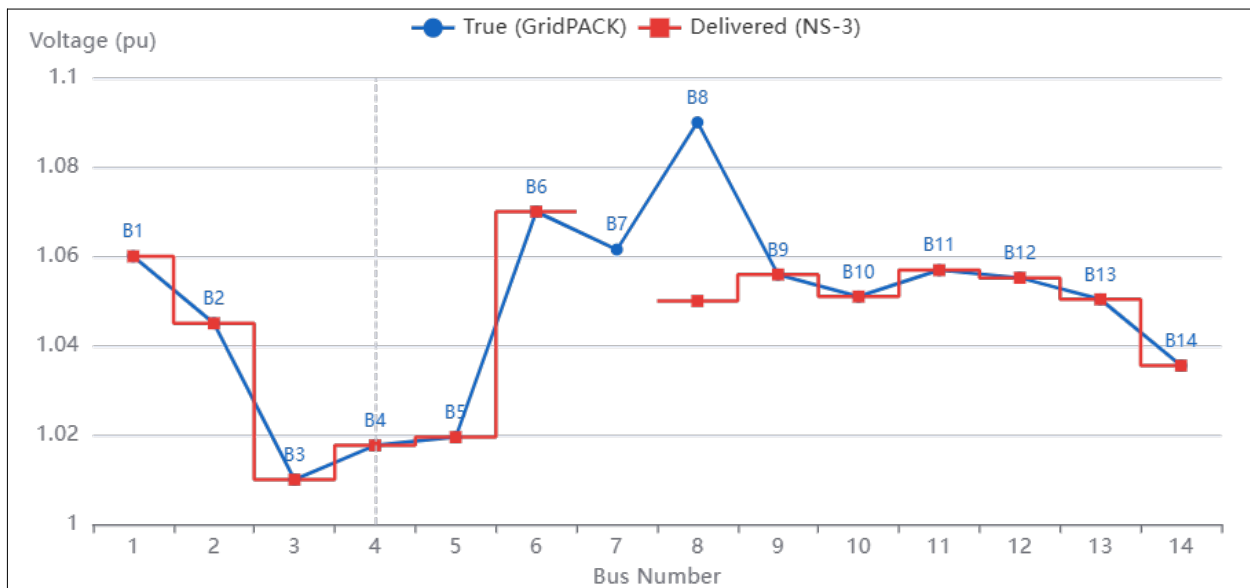
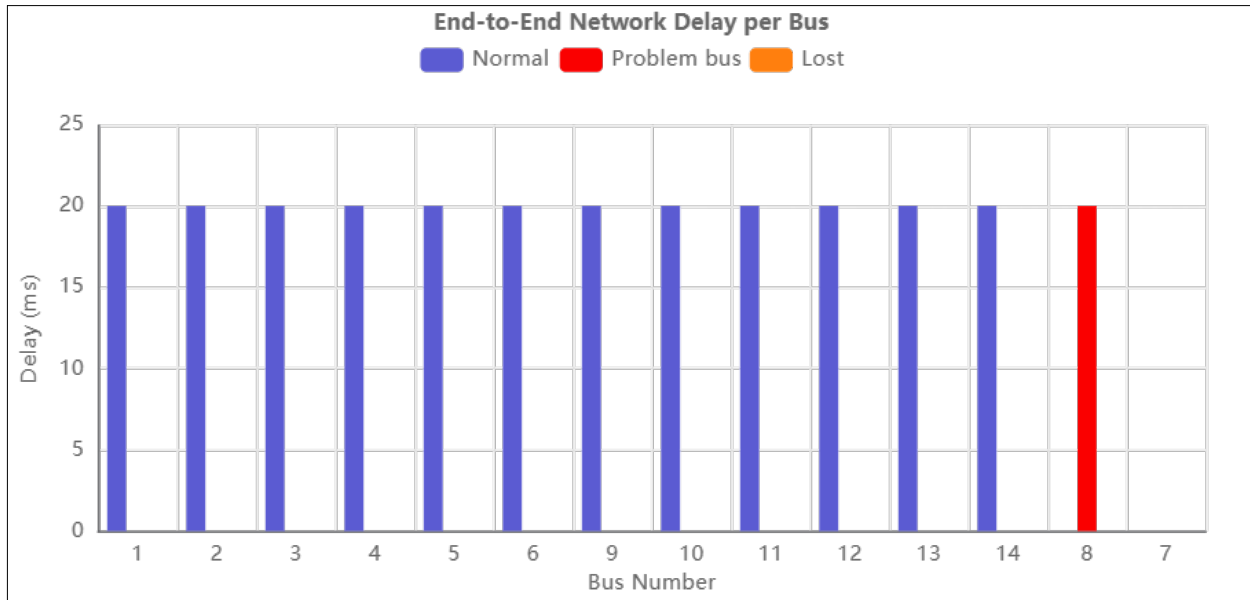


Fig 12: The complete operator dashboard under cyber-attack operation

Each panel of the dashboard is shown individually below: the grid voltage comparison, the network delay, the network performance summary, and the per-bus measurement table.

Scenario Status

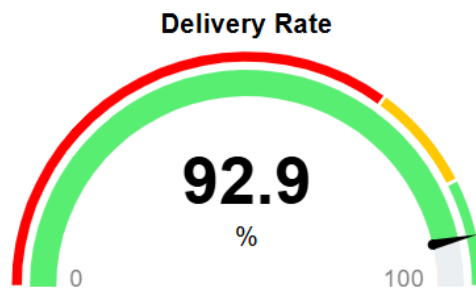
Scenario: combined | Latency: 20 ms | ● UNDER ATTACK | HELICS: connected (2 federates synced)



Network Performance

Delivered: 13/14 | Lost: 1 (7.1%) | Violations seen: 0 | Avg delay: 20.04 ms

⚠ ATTACK: FALSE DATA INJECTION (bus 8) — MISSED violations: 1



Per-Bus Measurement Table

Bus	True V	Recv V	Delay ms	Status
1	1.0600	1.0600	20.03	OK
2	1.0450	1.0450	20.03	OK
3	1.0100	1.0100	20.03	OK
4	1.0177	1.0177	20.04	OK
5	1.0195	1.0195	20.04	OK
6	1.0700	1.0700	20.03	OK
7	1.0615	—	—	LOST
8	1.0900	1.0500	20.03	⚠ TAMPERED
9	1.0559	1.0559	20.04	OK
10	1.0510	1.0510	20.04	OK
11	1.0569	1.0569	20.04	OK
12	1.0552	1.0552	20.04	OK
13	1.0504	1.0504	20.04	OK
14	1.0355	1.0355	20.04	OK

7. Extended Testbed: Multi-PMU Co-Simulation

The prototype described in Sections 2 to 6 demonstrated the complete cyber-physical data path with a single PMU on the IEEE 14-bus system. It had three important simplifications: the GridPACK federate replayed one saved power-flow solution rather than solving the network during a run; each HELICS time step carried a different bus, so there was no real time series; and the control command reached GridPACK but did not change the grid state. The extended testbed removes these simplifications and scales the study to larger systems. Its main additions are:

Live power flow. GridPACK now re-solves the power flow at every grid step with varying loads, and a control command changes a generator voltage setpoint that is applied in the next solution, so the monitoring-and-control loop is genuinely closed.

Selectable bus systems. The IEEE 14, 30, 39, 57, 118 and 300-bus systems are available, converted from the standard MATPOWER data into the PSS/E format that GridPACK reads.

Optimal PMU placement. PMUs are placed at the minimum set of buses that makes the whole network observable, computed by integer linear programming.

Realistic synchrophasor stream. Every PMU reports voltage and branch-current phasors at 30 frames per second over its own network link to a phasor data concentrator (PDC).

Control center. A new federate performs PMU state estimation with chi-square bad data detection and issues voltage control commands, which travel back over the simulated network.

Attacks and defenses. Simple and stealthy false data injection, packet drop and delay attacks can be launched against chosen PMUs, and the control center's detection and control response is measured.

Web application. Experiments can be configured, queued and analysed from a browser.

7.1 Architecture

The extended testbed consists of three federates synchronized by the HELICS broker (Fig. 13). The grid federate solves the power flow with GridPACK every 0.5 s and publishes the true voltage and current phasors seen by each PMU. The network federate (NS-3) samples those phasors at the PMU reporting rate, adds measurement noise, and sends each PMU's frame over its own point-to-point link to the PDC. The PDC time-aligns the frames and passes each complete measurement set to the control-center federate. Commands issued by the control center are sent over the network to the generator's node and handed to the grid federate, which applies them at its next power-flow solution. NS-3 advances one PMU frame (33 ms) per HELICS time step, so every message carries a consistent simulation time.

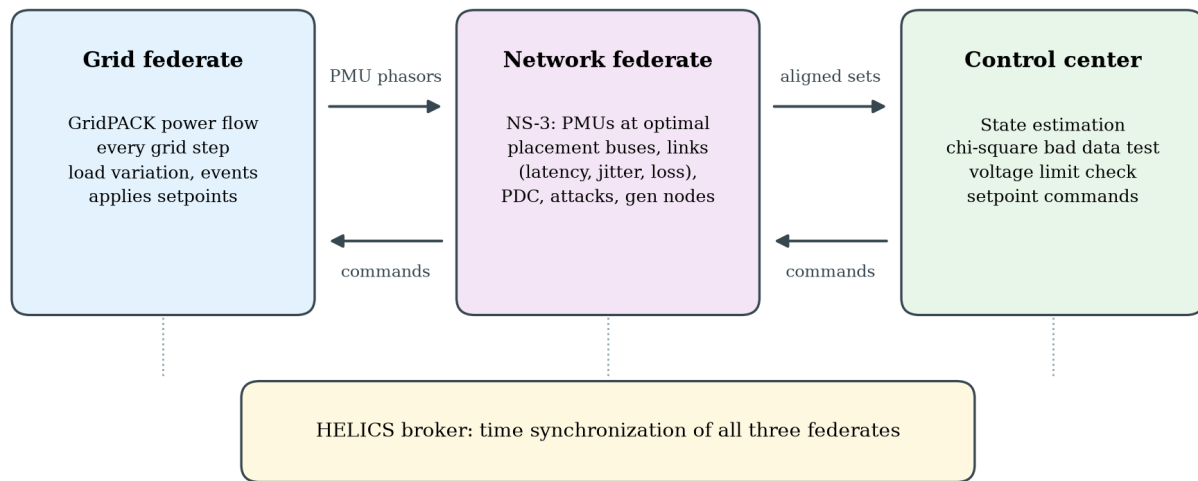


Fig 13: Architecture of the extended testbed: three HELICS federates with a closed monitoring-and-control loop.

7.2 Live GridPACK Power Flow and Bus Systems

GridPACK's power-flow application links against the PETSc numerical library, which was missing from the original environment; it was rebuilt (PETSc 3.16 with real scalars, SuperLU_DIST, ParMETIS) so that GridPACK runs again inside the container. At each grid step the grid federate writes the network with the current loads and generator setpoints to a PSS/E v23 file, runs GridPACK, and reads back the solution. Loads vary slowly ($\pm 2\%$ sinusoid with a 20 s period plus a small random walk), and scheduled events such as a generator voltage regulator (AVR) setpoint fault or a load step can be injected.

To validate the solver chain, GridPACK was compared with an independent Newton-Raphson implementation on all six systems (Table 5). The maximum voltage difference is below 5×10^{-7} pu in every case, which is the rounding of GridPACK's output, and one solution takes 0.34–0.43 s including process start-up.

System	Buses	Branches	Generators	GridPACK iterations	Max $ \Delta V $ vs. reference (pu)
IEEE 14	14	20	5	3	4.8×10^{-7}
IEEE 30	30	41	6	3	4.5×10^{-7}
IEEE 39	39	46	10	4	4.9×10^{-7}
IEEE 57	57	80	7	4	5.0×10^{-7}
IEEE 118	118	186	54	4	4.8×10^{-7}
IEEE 300	300	411	69	5	5.0×10^{-7}

Table 5: GridPACK power-flow validation against an independent Newton-Raphson solver.

7.3 Optimal PMU Placement

A PMU at bus i measures the voltage phasor of bus i and the current phasors of every branch connected to it, which makes bus i and all of its neighbours observable. The minimum placement solves the integer linear program: minimise the number of PMUs subject to $Ax \geq 1$, where A is the bus connectivity matrix and x marks the buses that receive a PMU. A redundant variant requires every bus to be observed by at least two PMUs ($Ax \geq 2$), so that the grid stays observable when any single PMU is lost or compromised. The minimum PMU counts in Table 6 agree with those reported in the literature for these systems.

System	Minimum PMUs	Share of buses	Redundant placement (each bus seen twice)
IEEE 14	4	29%	9
IEEE 30	10	33%	21
IEEE 39	13	33%	28
IEEE 57	17	30%	33
IEEE 118	32	27%	68
IEEE 300	87	29%	202

Table 6: Optimal PMU placement for full observability.

7.4 Communication Network

Each PMU is an NS-3 node with its own 10 Mbps point-to-point link to the PDC at the control center. Link latencies are drawn around a configurable mean ($\pm 50\%$ by default), each frame incurs 2 ms of processing plus random jitter (0–2 ms), and packet loss can be applied to every link. PMU measurements carry Gaussian noise of 0.1% in magnitude and 0.001 rad in angle. The PDC releases a time-aligned measurement set as soon as all PMUs have reported for that timestamp, or 20 ms after the first frame arrived; frames arriving later are counted as late. Frames are encoded as JSON messages carrying the phasors, rather than binary IEEE C37.118 frames.

Attacks are applied at the egress of the targeted PMUs, modelling a compromised substation network. False data injection (FDI) changes the reported phasors: the simple variant falsifies only the one channel that most directly measures the target bus, while the stealthy variant changes every channel connected to the target bus consistently with the network model ($a = Hc$), recomputing the change from the live measurements in every frame. The drop attack discards the targeted PMUs' frames and the delay attack holds them back by a configurable time.

7.5 Control Center: State Estimation, Bad Data Detection and Control

Because PMUs measure complex voltages and currents directly, the measurement model is linear, $z = H V + e$, and the weighted least-squares estimate is $\hat{V} = (H^H W H)^{-1} H^H W z$. The residual statistic $J = \sum |r_k|^2 / \sigma_k^2$ follows a chi-square distribution with $2m - 2n$ degrees of freedom for clean data (m measurements, n buses); J above the 99% threshold raises a bad data alarm. On an

alarm the PMU with the largest normalized residual is removed and the state is re-estimated, up to three PMUs. If the remaining measurements do not make the network observable, the previous estimate is used for the unobserved buses.

The estimated voltages are checked against the limits 0.94–1.08 pu. For a violating bus, the control center lowers (or raises) the voltage setpoint of the nearest generator by 0.01 pu, with a 0.003 pu deadband to avoid reacting to noise and a 1 s hold-off per generator. Control is suspended while an alarm cannot be cleaned by removing PMUs, so that the operator does not act on data known to be falsified.

7.6 Web Application

A web application gives remote users access to the testbed, in line with the project goal of a remotely accessible CPS testbed. Users select a bus system, simulation length, PMU placement, network conditions, grid event, attack and defenses, or start from presets, and submit the experiment to a queue that runs one co-simulation at a time. For each finished run the browser shows summary metrics and charts of the true and estimated voltage at any bus, the chi-square statistic, PMU latency, per-PMU delivery and PDC set completeness, together with the list of control commands. All inputs, logs and per-frame records of each run are stored for later analysis. The application is protected by a login and is reached through an SSH tunnel.

7.7 Deployment and Container Images

The complete testbed is packaged as a public container image at ghcr.io/mazharulmd/cps-testbed, and the source code, bus-system cases and build files are in the GitHub repository github.com/mazharulmd/cps-testbed. Four image tags are published:

latest: the current full testbed, with live GridPACK, the IEEE 14–300 bus cases, the multi-PMU NS-3 network, the control center, the web application (port 8080) and the Node-RED dashboard (port 1880). This is the tag to use for running the testbed.

v2.1: the current release under a fixed name. It runs the same simulation code as v2 and adds the redesigned web application (project header, colour-coded results, phone layout), validation of event and attack buses against the chosen bus system, and automatic opening of finished runs.

v2: the multi-PMU release under a fixed name that does not change when latest is updated. All results in Section 8 were produced with this version, so it should be used to reproduce them.

base: the earlier prebuilt image containing the compiled HELICS, GridPACK and NS-3 and the single-PMU demo. It is the starting point from which the repository's Dockerfile builds the full testbed (compiling PETSc for GridPACK and the NS-3 federate), and is not meant to be run directly.

Any machine with Docker can start the testbed with one command:

```
docker run -d -p 1880:1880 -p 8080:8080 -e CPS_WEB_PASSWORD=<password>
ghcr.io/mazharulmd/cps-testbed:v2
```

The web application is then available at <http://localhost:8080>. Logins for the web application and the Node-RED dashboard are supplied through environment variables, so no password is stored in the image.

8. Extended Results

All experiments below simulate 10 s with a 0.5 s grid step, 30 PMU frames per second, 10 ms mean link latency, a 20 ms PDC wait window and minimum PMU placement unless stated otherwise. Each run's scenario is reproducible from its stored configuration and random seed.

8.1 Scalability Across Bus Systems

Table 7 shows the baseline (no attack, no loss) on five systems. GridPACK converged at every grid step, all frames were delivered, and the state estimate stayed within 0.0007 pu of the true voltages with no false alarms. The run time grows with the number of PMUs and remains practical for interactive use: a 118-bus experiment finishes in about 40 s and a 300-bus experiment with 87 PMUs and 26,000 frames in under 80 s.

System	PMUs	Frames	Delivered	Mean latency (ms)	p95 latency (ms)	State estimation error (pu)	Run time (s)
IEEE 14	4	1,196	100%	14.1	17.7	0.00063	9.2
IEEE 30	10	2,990	100%	12.8	17.7	0.00026	9.4
IEEE 57	17	5,083	100%	12.6	17.7	0.00049	14.0
IEEE 118	32	9,568	100%	12.7	17.8	0.00024	40.5
IEEE 300	87	26,013	100%	13.1	17.3	0.00033	78.1

Table 7: Baseline performance on five bus systems (10 s of simulated time).

8.2 Closed-Loop Voltage Control

On the IEEE 118-bus system, a fault in the automatic voltage regulator of generator 49 raises its setpoint to 1.12 pu at $t = 4$ s, pushing bus 49 above the 1.08 pu limit (Fig. 14). Without control the over-voltage persists for the remaining 6.5 s. With control enabled the control center detects the violation in the first measurement set after the fault, issues a command to lower generator 49's setpoint at $t = 4.067$ s, the command crosses the network in 40 ms, and GridPACK applies it at the next grid step ($t = 4.5$ s). The violation lasts one grid step (0.5 s), and the estimated voltage tracks the true voltage throughout.

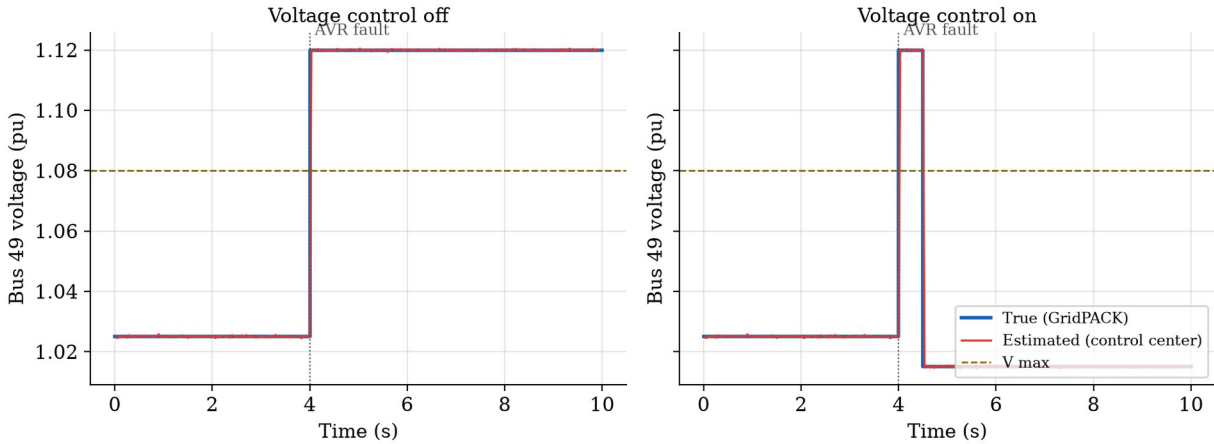


Fig 14: Bus 49 voltage after a generator AVR fault on the IEEE 118-bus system, without and with voltage control.

8.3 Network Latency and the PDC Wait Window

Table 8 varies the mean link latency with the PDC wait window fixed at 20 ms. At 5 ms every measurement set is complete. At 20 ms the latencies of different PMUs spread over 10–30 ms, wider than the wait window, so a quarter of the sets are released before the slowest PMUs arrive. At 50 ms and above no set is complete: the PDC releases each set 20 ms after its first frame, the remaining frames arrive late, and the control center falls back on stale estimates (error 0.010–0.018 pu instead of 0.0002 pu). The wait window must therefore cover the spread of PMU latencies, trading data completeness against end-to-end delay.

Mean link latency (ms)	Mean latency of frames in time (ms)	Frames in time	Complete PDC sets	State estimation error (pu)
5	7.7	100%	100%	0.00024
20	22.7	99%	76%	0.00025
50	40.1	45%	0%	0.01009
100	65.8	19%	0%	0.01767

Table 8: Effect of link latency with a fixed 20 ms PDC wait window (IEEE 118-bus).

8.4 Attacks and Defenses

Table 9 combines the generator 49 AVR fault at $t = 4$ s with attacks on the PMUs observing bus 49 between $t = 4$ s and $t = 9$ s. The attacker's goal is to hide the over-voltage by making the control center see 1.02 pu.

Scenario	Compromised PMUs	Detected by bad data test	Violation time (s)	Outcome
Voltage control off	—	—	6.5	Violation persists to the end of the run
No attack	—	—	0.5	Corrected by one setpoint command

Scenario	Compromised PMUs	Detected by bad data test	Violation time (s)	Outcome
5% packet loss	–	–	0.5	Only 18% of PDC sets complete, but the violation is still seen and corrected
Simple FDI	49	Yes, within 33 ms	0.5	PMU 49 removed; control acts on the cleaned estimate
Stealthy FDI	45, 49	No	0.5	Bus 49 appears at 1.02 pu, but neighbouring bus 48 (1.097 pu) reveals the over-voltage
Delay attack (100 ms)	45, 49	No	5.5	Frames miss the PDC window; bus 49 unobserved until the attack ends
Drop attack	45, 49	No	5.5	Same effect as the delay attack

Table 9: Attacks during a generator AVR fault on the IEEE 118-bus system.

The simple attack changes one channel inconsistently with the rest of the measurements, so the chi-square statistic jumps by more than an order of magnitude above the threshold in the first attacked set (Fig. 15, left); the falsified PMU is identified and removed and control proceeds. Once the fault is corrected the attack has nothing left to hide and its residual falls back to the noise level. The stealthy attack changes both PMUs that observe bus 49 consistently with the network model, so the statistic never moves (Fig. 15, right) and bus 49 is hidden. It still fails here because the physical coupling raises the voltage of the neighbouring bus 48 above the limit, which the control center corrects by acting on the same generator: hiding one bus is not enough, and a successful attacker must falsify the whole affected region.

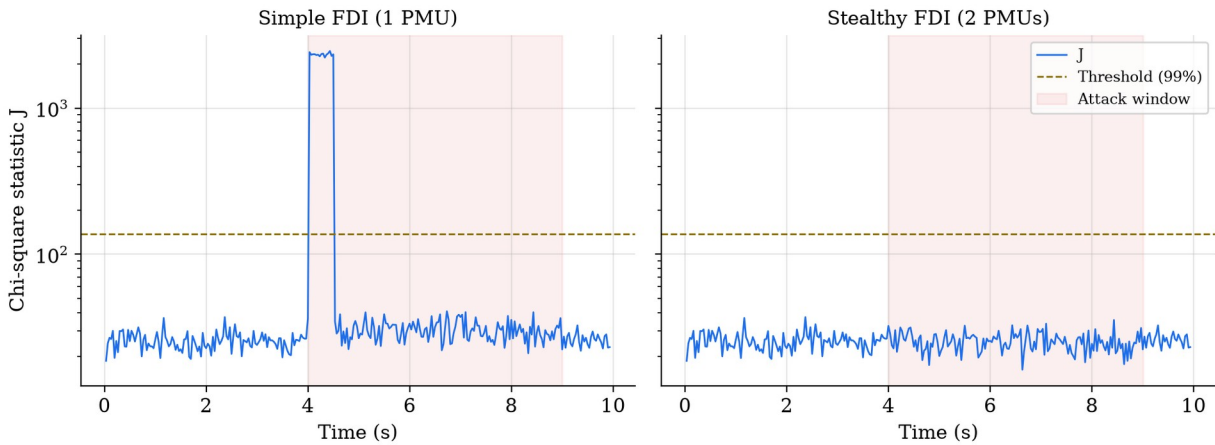


Fig 15: Chi-square bad data statistic under simple and stealthy false data injection (IEEE 118-bus).

Availability attacks are not visible to the bad data test at all. Delaying the two PMUs around bus 49 by 100 ms makes their frames arrive after the PDC has released each set (Fig. 16, right). Bus 49 becomes unobserved, the control center keeps its last estimate of 1.025 pu, and the over-voltage goes unnoticed for 5 s until the attack stops, after which it is corrected within one grid step (Fig. 16,

left). Dropping the frames has exactly the same effect. These attacks call for monitoring of PMU availability and latency in addition to data integrity checks.

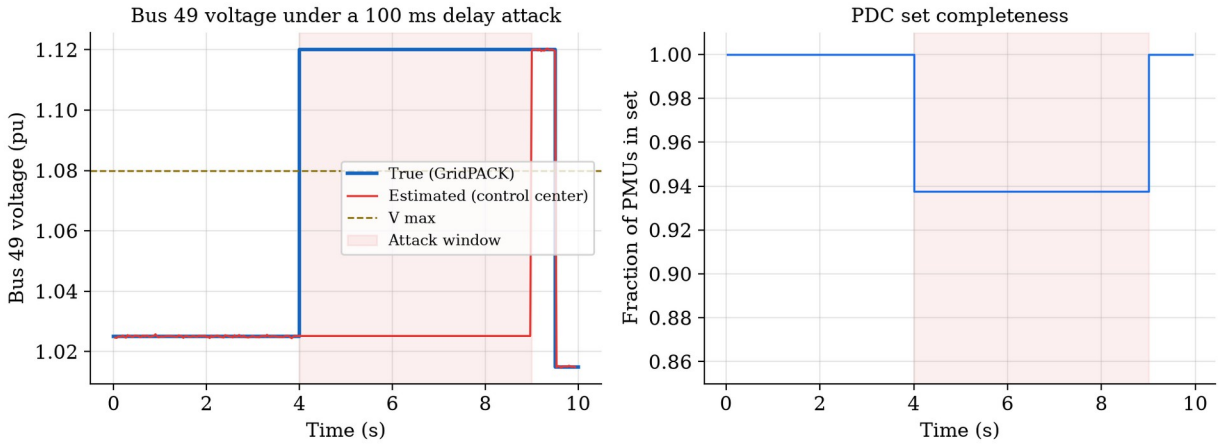


Fig 16: Delay attack on the PMUs observing bus 49: bus voltage (left) and PDC set completeness (right).

8.5 Critical Measurements and Redundant PMU Placement

With the minimum placement on the IEEE 14-bus system (PMUs at buses 2, 7, 11 and 13), bus 8 is observed only through the current of branch 7–8 measured at bus 7. Such a critical measurement has no redundancy, so falsifying it cannot be detected by any residual test. A simple FDI on that single channel makes the control center see 1.05 pu at bus 8 while the true voltage is 1.09 pu, with no alarm (Fig. 17, left). With the redundant placement (9 PMUs, including one at bus 8), the same simple attack is detected in every attacked measurement set, the falsified PMU is removed, and the estimate stays at the true 1.09 pu (Fig. 17, right). Minimum placement is sufficient for observability but leaves blind spots for bad data detection; redundant placement removes them at roughly twice the number of PMUs.

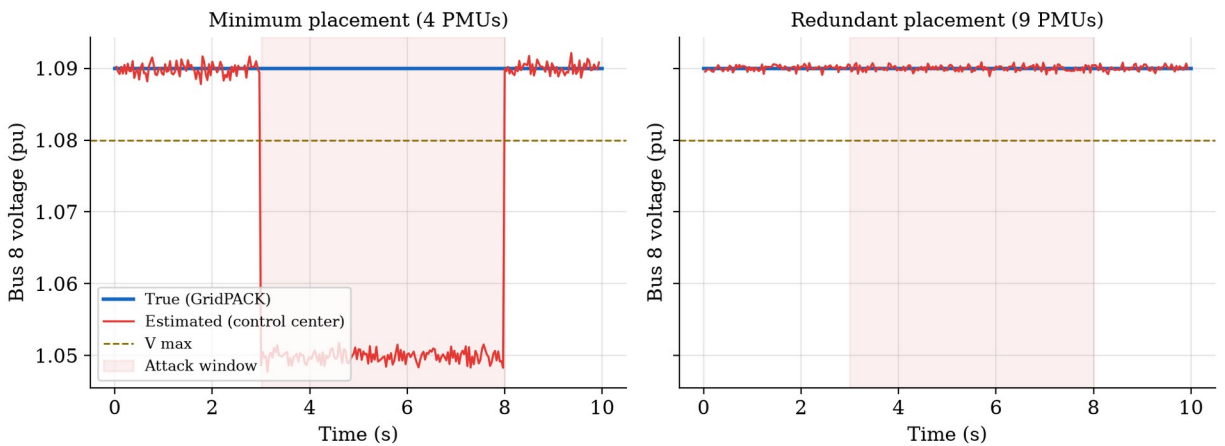


Fig 17: Simple false data injection on bus 8 of the IEEE 14-bus system with minimum and redundant PMU placement.

9. Summary

This work integrates four components - the GridPACK power-system simulator, the NS-3 communication-network simulator, the HELICS time-synchronization engine, and a Node-RED operator dashboard into a single, controllable co-simulation of the IEEE 14-bus system. The two simulators exchange data only through HELICS, which keeps them synchronized in time, while the dashboard presents each run's results to an operator.

Across three scenarios, the testbed demonstrates the full range of behavior it was built to study. In normal mode it transports every measurement faithfully, with end-to-end delay that tracks the configured network latency, confirming that the communication layer genuinely shapes the data path. Under packet loss it reproduces a communication-layer failure, in which a healthy bus becomes a blind spot because its measurement never arrives. Under a false-data-injection attack it reproduces a cyber-security failure, in which a genuine voltage violation is hidden from the control center by a forged measurement, defeating the protection scheme silently.

Together these results show a working cyber-physical testbed that can distinguish a healthy data path, a network fault, and a data-integrity attack - and can reproduce each safely and repeatably. This provides a controlled environment in which synchrophasor monitoring and grid cyber-security scenarios can be examined without any risk to the live national grid.

The extended testbed turns the prototype into a closed-loop, multi-PMU co-simulation that scales from the IEEE 14-bus to the IEEE 300-bus system. GridPACK solves the network live at every grid step and its results agree with an independent solver to within 5×10^{-7} pu; PMUs are placed optimally for observability; NS-3 carries 30 frames per second from every PMU to the PDC under configurable latency, jitter, loss and attacks; and a control-center federate estimates the state, tests for bad data and corrects voltage violations through commands that travel back over the network.

The experiments show that closed-loop control clears a generator voltage fault within one grid step; that the PDC wait window must match the spread of PMU latencies or measurement sets become incomplete and estimates stale; that chi-square bad data detection catches simple false data injection within one frame but not stealthy injection, which only fails when neighbouring buses reveal the violation; that delay and drop attacks defeat monitoring without triggering any integrity check; and that minimum PMU placement leaves critical measurements that can be falsified undetected, a blind spot removed by redundant placement.

The current implementation has limitations that define the next steps: the power system is quasi-steady-state rather than dynamic, frames are JSON rather than binary IEEE C37.118, the network is a star of PMU links around a single PDC, frames are not authenticated, and the stealthy attacker hides a single bus. Natural extensions are GridPACK dynamic simulation for oscillation studies, hierarchical PDC architectures, message authentication and latency-aware PDC policies, and multi-bus stealthy attacks with corresponding detectors.